

Seguridad en el Despliegue de Sistemas LLM

Módulo 9 · Infraestructura y Despliegue

Autenticación, rate limiting, guardrails, enmascaramiento PII, auditoría de logging y checklist de seguridad completo.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Seguridad en el Despliegue de Sistemas LLM

Los sistemas LLM en producción presentan una superficie de ataque única que combina vulnerabilidades de software tradicional con vulnerabilidades específicas de los modelos de lenguaje. El OWASP Top 10 para aplicaciones LLM (2025) documenta las diez vulnerabilidades más críticas, con prompt injection en el primer lugar. Un sistema LLM mal asegurado puede: revelar datos confidenciales de otros usuarios, ejecutar acciones no autorizadas si tiene acceso a herramientas, consumir recursos de forma descontrolada (cost amplification attacks), o ser utilizado para generar contenido dañino a escala.

La seguridad en el despliegue de LLMs tiene dos dimensiones: la seguridad de la infraestructura (cómo se protege el sistema de accesos no autorizados, vulnerabilidades de red, y fallos de configuración) y la seguridad del modelo (cómo se protege el modelo de manipulaciones a través del prompt y cómo se garantiza que el modelo se comporta de acuerdo con las políticas de la organización). Ambas dimensiones son necesarias para un sistema seguro.

Las dimensiones de la seguridad en sistemas LLM

Autenticación y autorización

La autenticación verifica quién eres (API key, JWT, OAuth 2.0). La autorización verifica qué puedes hacer (RBAC: un usuario de nivel free no puede usar modelos premium, un empleado de soporte no puede acceder a datos de clientes de otros equipos). Para sistemas multi-tenant, cada petición debe verificar que el usuario tiene acceso al modelo, a los documentos del corpus RAG relevantes, y a las herramientas que solicita usar. La separación de datos entre tenants a nivel de retrieval (no a nivel de generación) es crítica para sistemas RAG compartidos.

Secretos y configuración segura

Los secretos (API keys de proveedores de modelos, credenciales de bases de datos, claves de cifrado) nunca deben estar hardcoded en el código o en las imágenes Docker. Las mejores prácticas: almacena secretos en Kubernetes Secrets (base64-encoded, con acceso controlado por RBAC), usa HashiCorp Vault o AWS Secrets Manager para secretos de alta criticidad con rotación automática, referencia secretos desde variables de entorno en el pod (env.valueFrom.secretKeyRef), y escanea las imágenes Docker con herramientas como Trivy o Snyk para detectar secretos filtrados en las capas de la imagen.

Network security: segmentar y controlar el tráfico

Los pods de inferencia LLM nunca deben ser accesibles directamente desde internet. La arquitectura correcta: los pods de inferencia están en una subnet privada sin acceso a internet directo. Solo el API Gateway tiene acceso público. Los Network Policies de Kubernetes restringen qué pods pueden comunicarse con qué otros pods (por ejemplo, solo el API Gateway puede comunicarse con los pods de inferencia). mTLS (mutual TLS) entre microservicios

garantiza que solo los servicios autorizados se pueden comunicar entre sí.

SECCIÓN 3 · CÓMO FUNCIONA

Protección del modelo: guardrails y validación de outputs

Input guardrails: validar antes de enviar al modelo

Los input guardrails son filtros que se aplican al input del usuario antes de enviarlo al LLM. Los tipos más comunes: detección de prompt injection (heurísticas que detectan intentos de override de instrucciones del sistema), filtrado de contenido (detectar solicitudes de contenido inapropiado usando modelos clasificadores específicos), validación de longitud (rechazar prompts que excedan el límite de tokens sin procesar), y sanitización de PII (enmascarar datos personales antes de enviarlos al modelo). Herramientas: Guardrails AI, NeMo Guardrails (NVIDIA), LlamaGuard (Meta).

Output guardrails: validar antes de devolver al usuario

Los output guardrails validan la respuesta del modelo antes de devolverla al usuario. Verifican que: la respuesta no contiene información que no debería (PII de otros usuarios, secretos del sistema), la respuesta es faithful al contexto (para sistemas RAG), la respuesta no contiene contenido dañino o inapropiado, y la respuesta sigue el formato esperado (para sistemas que requieren JSON estructurado). Las herramientas de output validation incluyen: Guardrails AI con validadores predefinidos, evaluación con LLM-as-judge para políticas complejas, y regex/schema validation para outputs estructurados.

Audit logging: registrar todo para cumplimiento

En sectores regulados (salud, finanzas, legal), el audit logging es un requisito legal además de una buena práctica. Los logs de auditoría deben incluir: timestamp, usuario/sesión que realizó la petición, modelo y versión usada, tokens de entrada y salida, la respuesta completa (o un hash para verificabilidad sin almacenamiento de datos sensibles), y el resultado de los guardrails aplicados. Los logs deben almacenarse de forma inmutable (no modificables tras su creación) y retenirse según los requisitos regulatorios (típicamente 1-7 años).

SECCIÓN 4 · EJEMPLOS REALES

- Sistema de seguridad en capas de Harvey AI (legal): todas las peticiones pasan por input guardrails que detectan intentos de extraer datos de otros clientes, autenticación con JWT con claims de tenant_id para aislamiento de datos de forma garantizada en el retrieval, output guardrails que verifican que las respuestas no contienen información de otros expedientes, y audit logging de todas las interacciones con retención de 7 años para cumplimiento de regulaciones legales.
- Protección contra cost amplification attacks (empresa cloud): un atacante descubrió que enviando prompts de 100.000 tokens podía consumir \$100 de cómputo con un solo request. Solución: rate limiting por TPM (no solo RPM), límite máximo de tokens por request en el API Gateway (rechaza cualquier request con input > 10.000 tokens para usuarios de plan Free), y circuit breaker que desactiva temporalmente la API de un usuario cuando detecta patrones anómalos.

- Enmascarado de PII con NLP antes del LLM (banco): todos los inputs de clientes pasan por un NLP classifier que detecta PII (nombres, números de cuenta, fechas de nacimiento, etc.) y los enmascara con pseudónimos antes de enviarlos al LLM. El modelo trabaja con el texto enmascarado y la respuesta se de-pseudonimiza antes de devolverla al usuario. El LLM nunca procesa datos personales reales del cliente.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Confiar en que el system prompt es suficiente para el control de acceso. El system prompt diciendo "No reveles datos de otros usuarios" no es un control de seguridad fiable: puede ser bypassado por prompt injection. El control de acceso real está en el retrieval (el usuario solo recupera documentos a los que tiene acceso) y en la separación de contextos a nivel de arquitectura, no en instrucciones del modelo.

Error 2: No cifrar los modelos almacenados en disco. Los pesos de modelos fine-tuneados con datos propietarios son activos de propiedad intelectual de alto valor. Almacenar estos modelos sin cifrado en discos accesibles expone la IP de la organización. Los modelos deben almacenarse cifrados y con acceso controlado.

Error 3: Logs de auditoría sin integridad garantizada. Un log que puede ser modificado no es un log de auditoría válido para cumplimiento regulatorio. Implementa logs de auditoría con cadenas de hash o en sistemas de almacenamiento inmutable (AWS CloudTrail, Azure Monitor) que garanticen que los registros no pueden ser alterados.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Checklist de seguridad para el despliegue de sistemas LLM: autenticación (API keys con hash, JWT con claims de tenant). Rate limiting (por RPM + TPM, máximo de tokens por request). Segregación de red (pods de inferencia en subnet privada, Network Policies en Kubernetes). Secretos en Vault o Kubernetes Secrets (no en código ni en imágenes). Escaneo de imágenes con Trivy. Input guardrails (detección de injection, filtrado de contenido, límite de longitud). Output guardrails (validación de formato, faithfulness para RAG). Audit logging inmutable con retención adecuada. Cifrado de modelos en reposo. Enmascarado de PII antes de procesamiento por LLM.

Para implementar guardrails básicos con Guardrails AI: `pip install guardrails-ai`. Define las validaciones en un Rail spec (YAML o Python). Wrap el LLM con el guard: `guard = Guard().use(NoHarmfulContent(), OnTopic(valid_topics=["support", "product"]))`. `guard.parse(llm_output=llm_response)`. Si la validación falla, guard puede reintentar la llamada al LLM con instrucciones adicionales o devolver un output de fallback predefinido.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M12 (Seguridad en IA): profundiza en los ataques específicos (prompt injection, jailbreaks) y las mitigaciones.
- M6-T7 (Prompts robustos): las defensas a nivel de prompt son complementarias a las defensas de infraestructura.
- M13 (Gobernanza): el audit logging y el cumplimiento regulatorio son requisitos de gobernanza.

La seguridad de sistemas LLM en producción tiene dos dimensiones: infraestructura (autenticación JWT/RBAC, segregación de red, secretos en Vault, escaneo de imágenes con Trivy) y modelo (input guardrails contra injection y contenido dañino, output guardrails para validación, audit logging inmutable, enmascarado de PII). El aislamiento de tenants en RAG debe ser a nivel de retrieval (datos, no instrucciones). La defensa en profundidad requiere todas las capas simultáneamente. Herramientas: Guardrails AI, NeMo Guardrails, LlamaGuard (Meta). El system prompt no es un control de seguridad fiable: el control real está en la arquitectura.